

Nexus Grid: Firewall Breach

Difficulty: Medium **Tags:** Graphs, BFS, Multi-source BFS, Matrix, Simulation

Story

You're a security engineer at **Nexus Systems**, monitoring a data-center laid out as a grid of server nodes. To survive random hardware failures, every node is cross-wired to its **8 surrounding neighbors** — not just up/down/left/right, but the 4 diagonals too, forming a full mesh for redundancy.

Late one night, an attacker plants a worm on a handful of nodes. Because of the mesh wiring, the worm doesn't just crawl in straight lines — it can also jump across diagonal links. Every tick of the clock, every **compromised** node breaches all 8 of its neighboring **secure** nodes simultaneously.

Your job: simulate the breach and figure out how long the attacker needs to compromise the entire reachable network — or whether parts of it are safe forever.

Problem Statement

You are given a grid (`grid`) of size (`n x m`). Each cell has one of three states:

- `0` — **Dead slot**: no server is racked here (empty, inert — never gets infected, never spreads infection).
- `1` — **Secure node**: a healthy, uninfected server.
- `2` — **Compromised node**: an already-infected server, actively spreading the breach.

Every **tick** (unit of time), every compromised node infects **all secure nodes among its 8 neighbors** (horizontal, vertical, and both diagonals) — all infections happen simultaneously across the whole grid before the next tick begins.

Return the **minimum number of ticks** required until no secure node (`1`) remains in the grid.

If there exists at least one secure node that can **never** be reached by the breach (no matter how many ticks pass), return `-1`.

Function Signature

```
int minTicksToBreach(vector<vector<int>>& grid);
```

(Python: `def minTicksToBreach(self, grid: List[List[int]]) -> int:`)

Input Format

```
T                // number of test cases
n m              // rows and columns of the grid
grid[0][0] ... grid[0][m-1]
grid[1][0] ... grid[1][m-1]
```

```

...
grid[n-1][0] ... grid[n-1][m-1]
````//
(repeated for each of the T test cases)

Output Format

For each test case, print a single integer — the minimum number of ticks,
or -1.

Constraints

...
1 <= T <= 50
1 <= n, m <= 500
grid[i][j] ∈ {0, 1, 2}

```

## Examples

### Example 1 — Straightforward diagonal spread

Input grid:

```

2 1 0
1 1 0
0 0 1

```

Output: **2**

Explanation:

- Tick 0 (start): compromised = {(0,0)}. Secure nodes: (0,1), (1,0), (1,1), (2,2).
- Tick 1: (0,0) breaches all 8 neighbors that are secure → (0,1), (1,0), (1,1) all get compromised (note (1,1) is a *diagonal* neighbor of (0,0)).
- Tick 2: (1,1) — now compromised — breaches its diagonal neighbor (2,2), the last secure node.
- All secure nodes compromised in **2 ticks**.

### Example 2 — Unreachable nodes (**-1**)

Input grid:

```

2 0 1
0 0 0
1 0 0

```

Output: **-1**

**Explanation:** (0,2) is 2 columns away from (0,0) — not adjacent even diagonally (adjacency only covers 1-cell distance in any of the 8 directions). Similarly (2,0) is 2 rows away from (0,0). Both secure nodes are permanently isolated, so the answer is **-1**.

### Example 3 — Already fully compromised

Input grid:

```
2 2
2 2
```

Output:  $\boxed{0}$

**Explanation:** There are no secure nodes ( $\boxed{1}$ ) anywhere, so zero ticks are needed.

---

### Example 4 — No attacker present

Input grid:

```
1 1
1 1
```

Output:  $\boxed{-1}$

**Explanation:** There isn't a single compromised node to start the breach, so the secure nodes can never be infected — even though they're all connected to each other.

---

### Example 5 — Why 8 directions matter (diagonal-only reachability)

Input grid:

```
2 0 0
0 1 0
0 0 0
```

Output:  $\boxed{1}$

**Explanation:** (1,1) is only reachable from (0,0) **diagonally** — under a 4-directional rule this node would be unreachable ( $\boxed{-1}$ ), but since Nexus Grid mesh-wiring allows diagonal breach, it takes just **1 tick**.

---

### Example 6 — Mixed dead slots don't block diagonal spread

Input grid:

```
2 0 0 0
0 0 0 0
0 0 1 0
```

Output:  $\boxed{-1}$

**Explanation:** (2,2) is 2 rows and 2 columns from (0,0) — too far even diagonally (each tick only extends the breach by 1 cell in any of the 8 directions, it doesn't "skip" over dead slots). So it stays secure forever →  $\boxed{-1}$ .

---

### Edge Cases to Handle

1. Grid with a single cell ( $\boxed{1 \times 1}$ ) — either  $\boxed{[[1]]} \rightarrow \boxed{-1}$ ,  $\boxed{[[2]]} \rightarrow \boxed{0}$ , or  $\boxed{[[0]]} \rightarrow \boxed{0}$ .

2. Multiple compromised nodes scattered across the grid at the start (multi-source spread — all should tick simultaneously, not sequentially).
  3. A secure node fully surrounded by dead slots ( $\emptyset$ s) with no compromised node ever reachable.
  4. Very large grids (500 x 500) — solution must be efficient (aim for  $O(n*m)$ , not exponential simulation).
  5. A grid entirely made of dead slots ( $\emptyset$  only)  $\rightarrow$   $\emptyset$  ticks (nothing to infect).
- 

### Hint (only if you get stuck)

Think **multi-source BFS**, same core idea as the classic 4-directional version — just extend your direction array from 4 deltas to all 8:

```
{-1, -1}, {-1, 0}, {-1, 1},
{0, -1}, {0, 1},
{1, -1}, {1, 0}, {1, 1}
```

Push *all* initially compromised nodes into the queue at once, then process level by level (BFS layers = ticks), same as before.